

PostgreSQL + sqlc

Persistência type-safe para APIs Go

DIM0547 — Sprint 2 (05/05)

Prof. Fernando · UFRN · 2026.1

Hoje

O que vamos ver

- Por que `map` não serve para produção
- Por que sqlc em vez de ORM
- PostgreSQL com Docker
- Schema + queries → `sqlc generate`
- Conectar a API ao banco

O que fica para próxima aula

- Migrations (versionar o schema)
- Clean Architecture
- Interface de repositório
- Testes

0 problema: Sprint 1

```
var contacts = map[string]Contact{}
```

Três problemas fundamentais

1. Volatilidade — reinicia o servidor, perde tudo.

Em produção, containers reiniciam o tempo todo.

2. Race condition — Go executa handlers em goroutines paralelas.

`map` não é seguro na prevenção de condições de corrida.

3. Escala — load balancer → N instâncias → N mapas diferentes.

Banco de dados é o ponto de coordenação compartilhado.

As três abordagens

Abordagem	Exemplo	O que acontece na prática
ORM	GORM	Queries escondidas. Funciona até ficar lento — aí você depura SQL que não escreveu.
Raw SQL	<code>database/sql</code>	Controle total, SQL visível. Mas pode se tornar verboso e frágil.
SQL-first codegen	<code>sqlc</code> 	Você escreve SQL puro. O sqlc gera Go tipado. SQL visível + tipo checados em tempo de compilação.

Por que não ORM

```
// GORM – o que esse código faz?  
db.Where("email = ?", email).  
    Preload("Orders").  
    First(&contact)  
// Que SQL isso gera?  
// Quantas queries? Tem JOIN? Quantos?  
// E se ficar lento?
```

```
-- sqlc – você sabe exatamente o que acontece  
-- name: GetContactByEmail :one  
SELECT * FROM contacts WHERE email = $1;
```

ORMs escondem SQL. Quando a query fica lenta — e vai ficar — você vai debugar SQL de qualquer jeito. Melhor aprender SQL desde o começo. É uma habilidade transferível.

Por que sqlc especificamente

- **Zero reflexão em runtime** — GORM usa reflexão para mapear campos. sqlc gera código Go compilado sem intermediários em runtime.
- **Erros em tempo de geração** — se você errar o nome de uma coluna, `sqlc generate` falha. O erro aparece antes de chegar em produção.
- **Código legível** — `internal/db/contacts.sql.go` é Go normal. Você pode abrir, ler e entender. Não é caixa-preta.
- **SQL é a habilidade transferível** — o banco muda, o ORM muda, Go muda. SQL tem 50 anos e continua por aí.

Como sqlc funciona

Você escreve SQL		sqlc lê tudo		Você usa Go
<code>db/schema/001.sql</code> <code>CREATE TABLE ...</code>	→	sqlc generate	→	<code>internal/db/</code> <code>contacts.sql.go</code> <code>models.go</code> <code>db.go</code>
<code>db/queries/contacts.sql</code> <code>-- name: List :many</code> <code>SELECT * FROM ...</code>				

Erros de SQL são pegos em **tempo de geração**, não em tempo de execução na madrugada.

PostgreSQL com Docker

```
docker run -d --name lista04-postgres \  
  -e POSTGRES_PASSWORD=postgres \  
  -e POSTGRES_DB=lista04 \  
  -p 5432:5432 \  
  postgres:16-alpine
```

`-d` — background, não ocupa o terminal

`--name` — nome legível para `docker stop`

`-p 5432:5432` — `host:container`

`-e POSTGRES_*` — configuração inicial da imagem oficial

`postgres:16-alpine` — versão fixada.

Nunca use `latest` em projetos reais.

Falando com o banco: dois caminhos

Você tem duas formas de abrir um *console SQL* no container que está rodando:

A — `docker exec` sem instalar nada

```
docker exec -it lista04-postgres \  
psql -U postgres -d contacts
```

Usa o `psql` que **já vem dentro** da imagem do Postgres.
Funciona em qualquer máquina com Docker.

B — `psql` local precisa do cliente

```
psql "$DATABASE_URL"  
psql "$DATABASE_URL" -f schema.sql
```

Mais ergonômico — autocomplete, histórico do shell, redirecionar arquivos com `-f`. Mas exige instalar o **cliente** PostgreSQL.

Os comandos `psql "$DATABASE_URL" -f db/schema/*.sql` que aparecem na **Lista 4** assumem o caminho B.

Instalando o cliente psql

`psql` é só o **cliente** CLI. Não confunda com instalar um servidor PostgreSQL local — você não precisa, o servidor está no Docker.

```
# macOS
brew install libpq
brew link --force libpq
```

```
# Ubuntu / Debian
sudo apt install postgresql-client
```

```
# Fedora
sudo dnf install postgresql
```

Verificar:

```
psql --version
# psql (PostgreSQL) 18.x
```

Codespace já vem com `psql` configurado — esse slide importa só para quem trabalha **localmente**.

Por que Docker para o banco

"Docker garante que a versão é **idêntica** para todos: você, seu colega de equipe, o CI. Mesma imagem. Isso elimina a categoria inteira de bugs *'funciona na minha máquina'*."

```
# Verificar que está rodando  
docker ps
```

```
# URL de conexão – interface padrão entre aplicação e banco  
export DATABASE_URL="postgres://postgres:postgres@localhost:5432/contacts?sslmode=disable"  
  
# Caminho A: psql via docker exec (não precisa instalar nada)  
docker exec -it lista04-postgres psql -U postgres -d contacts  
  
# Caminho B: psql local (precisa do cliente – slide anterior)  
psql "$DATABASE_URL"
```

`sslmode=disable` apenas para desenvolvimento local. Em produção: sempre TLS.

Criando a tabela

```
CREATE TABLE contacts (  
  id          SERIAL          PRIMARY KEY,  
  name        TEXT           NOT NULL,  
  email       TEXT           NOT NULL UNIQUE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Tipo	Significado
SERIAL	Inteiro auto-incrementado. O banco atribui o ID — não a aplicação. Evita colisão em ambiente concorrente.
TEXT	String sem limite de tamanho. <code>VARCHAR(255)</code> é resquício do MySQL — no PostgreSQL moderno, use <code>TEXT</code> .
UNIQUE	Constraint no banco . Mesmo com requests simultâneos, o banco garante unicidade neste campo.
TIMESTAMPTZ	Timestamp com timezone, armazenado em UTC. Sempre use <code>TIMESTAMPTZ</code> , nunca <code>TIMESTAMP</code> .
DEFAULT NOW()	Banco preenche automaticamente — a aplicação não passa esse campo.

0 ciclo sqlc

Você escreve (humano)

```
db/schema/  
  001_contacts.sql  
  002_orders.sql  
  
db/queries/  
  contacts.sql  
  orders.sql  
  
sqlc.yaml
```

sqlc gera (máquina)

```
internal/db/  
  db.go          ← interface DBTX  
  models.go      ← structs tipados  
  contacts.sql.go  
  orders.sql.go
```

Você usa (humano)

```
queries.ListContacts(ctx)  
queries.CreateOrder(ctx, arg)
```

O sqlc lê o schema para inferir os tipos. Ele lê as queries para gerar as funções. Você nunca escreve código de acesso a banco — só SQL.

Anotações: a linguagem do sqlc

Todo comentário `-- name: X :tipo` instrui o sqlc sobre **como gerar** a função Go correspondente.

Anotação	Assinatura gerada	Caso de uso
<code>:many</code>	<code>(...) ([]T, error)</code>	SELECT sem LIMIT conhecido
<code>:one</code>	<code>(...) (T, error)</code>	SELECT por PK, INSERT com RETURNING
<code>:exec</code>	<code>(...) error</code>	DELETE, UPDATE sem retorno
<code>:execresult</code>	<code>(...) (pgconn.CommandTag, error)</code>	Quando precisa saber quantas linhas foram afetadas
<code>:copyfrom</code>	bulk insert	Inserir muitas linhas de uma vez (via COPY)

Se a query não encontrar nada com `:one`, o erro retornado é `pgx.ErrNoRows` — que você verifica no handler para retornar 404.

Anotações: exemplos variados

:many com filtro opcional

```
-- name: ListByStatus :many
SELECT * FROM orders
WHERE status = $1
ORDER BY created_at DESC;
```

:one com JOIN

```
-- name: GetOrderWithCustomer :one
SELECT o.*, c.name AS customer_name
FROM orders o
JOIN customers c ON c.id = o.customer_id
WHERE o.id = $1;
```

:exec para UPDATE

```
-- name: MarkAsShipped :exec
UPDATE orders
SET status = 'shipped',
    shipped_at = NOW()
WHERE id = $1;
```

:execresult quando importa saber

```
-- name: CancelPending :execresult
UPDATE orders
SET status = 'cancelled'
WHERE customer_id = $1
    AND status = 'pending';
-- rows affected = quantos pedidos foram cancelados
```

RETURNING: INSERT que devolve dados

Sem `RETURNING`, um INSERT não retorna nada — você não sabe o `id` gerado pelo banco.

```
-- Sem RETURNING — `:exec`  
-- name: CreateContactSilent :exec  
INSERT INTO contacts (name, email) VALUES ($1, $2);  
-- resultado: error. Você não sabe qual ID foi criado.  
  
-- Com RETURNING — `:one`  
-- name: CreateContact :one  
INSERT INTO contacts (name, email)  
VALUES ($1, $2)  
RETURNING *;  
-- resultado: Contact completo, incluindo id e created_at
```

`RETURNING` é uma extensão PostgreSQL — não existe em MySQL.

Parâmetros posicionais: \$1, \$2...

PostgreSQL usa parâmetros **posicionais**, não `?` como MySQL/SQLite.

```
-- MySQL / SQLite
SELECT * FROM contacts WHERE name = ? AND email = ?;

-- PostgreSQL
SELECT * FROM contacts WHERE name = $1 AND email = $2;
```

A vantagem: você pode referenciar o mesmo parâmetro múltiplas vezes.

```
-- name: SearchContacts :many
SELECT * FROM contacts
WHERE name ILIKE '%' || $1 || '%'
      OR email ILIKE '%' || $1 || '%';
-- $1 aparece duas vezes - impossível com ?
```

sqlc gera `SearchContactsParams` automaticamente com um único campo `Query string`.

Mapeamento de tipos SQL → Go

O sqlc lê o schema e infere os tipos Go correspondentes. Você nunca precisa declarar os structs.

Tipo PostgreSQL	Tipo Go gerado	Observação
SERIAL / INTEGER	int32	
BIGSERIAL / BIGINT	int64	Prefira para IDs em sistemas grandes
TEXT / VARCHAR	string	
BOOLEAN	bool	
NUMERIC / DECIMAL	pgtype.Numeric	Nunca float64 para dinheiro
TIMESTAMPTZ	time.Time	Sempre com timezone
UUID	pgtype.UUID ou [16]byte	
TEXT[]	[]string	Arrays nativos do PostgreSQL
JSONB	[]byte ou tipo customizado	

Colunas NOT NULL geram tipos diretos. Colunas nullable geram tipos com ponteiro ou pgtype.X — o sqlc força você a tratar o caso NULL explicitamente.

Nullable vs NOT NULL

```
CREATE TABLE products (  
    id          SERIAL PRIMARY KEY,  
    name        TEXT NOT NULL,  
    description TEXT,           -- nullable  
    deleted_at  TIMESTAMPTZ    -- nullable  
);
```

```
// models.go gerado  
type Product struct {  
    ID          int32  
    Name        string      // NOT NULL → tipo direto  
    Description pgtype.Text  // nullable → pgtype.Text{Valid: bool, String: string}  
    DeletedAt   pgtype.Timestampt  
}
```

Colunas nullable "sangram" para o código Go — você é **forçado** a lidar com o caso NULL. Isso é bom: evita `nil pointer dereference` em produção. Por isso declare `NOT NULL` sempre que possível no schema.

O que sqlc generate produz

```
internal/db/
├─ db.go           ← interface DBTX (aceita *pgx.Conn ou *pgxpool.Pool)
├─ models.go       ← um struct por tabela, tipos inferidos do schema
└─ contacts.sql.go ← uma função por query anotada
```

db.go define a interface que o `*Queries` usa internamente — isso permite passar tanto uma conexão direta quanto um pool, ou até uma transação (`pgx.Tx`).

models.go é gerado uma vez por tabela. Se você adicionar uma coluna ao schema e regenerar, o struct é atualizado automaticamente.

contacts.sql.go tem o SQL literal como constante Go e a função tipada. O SQL **não é interpretado em runtime** — é uma string constante passada ao banco.

Regra: nunca edite `internal/db/` à mão. Edite o `.sql`, rode `sqlc generate`, commite os dois juntos.

Pool de conexões

Cada request HTTP pode precisar de uma query ao banco.

Sem pool

request 1 → abre TCP
→ query
→ fecha TCP

request 2 → abre TCP
→ query
→ fecha TCP

Com pgxpool

request 1 → [conn 1] → banco

request 2 → [conn 2] → banco ← paralelo

request 3 → aguarda conn livre

Pool: min=2, max=4×CPUs (padrão)

Abrir uma conexão TCP tem custo fixo (~5ms). Com pool esse custo é pago uma vez. Sem pool, toda requisição sob carga paga esse custo.

`r.Context()` propaga o deadline do request HTTP para a query. Se o cliente desconectar, a query é **cancelada no banco** automaticamente — sem trabalho desperdiçado.

Estrutura do projeto Sprint 2

```
meu-projeto/
├── cmd/api/main.go
├── handler/
│   └── contacts.go          ← usa db.Queries
├── internal/
│   └── db/                  ← gerado pelo sqlc (não editar)
│       ├── contacts.sql.go
│       ├── db.go
│       └── models.go
├── db/
│   ├── schema/001_contacts.sql ← fonte da verdade do schema
│   └── queries/contacts.sql    ← queries anotadas
├── sqlc.yaml
├── docker-compose.yml        ← opcional (substitui o `docker run`)
└── go.mod
```

Mesma estrutura usada na **Lista 4**. `internal/` é convenção do Go: pacotes ali não podem ser importados por outros módulos — protege a API gerada do sqlc.

0 que vem na próxima aula

Migrations — versionar o schema

```
db/migrations/  
├─ 001_create_contacts.sql    # CREATE TABLE  
├─ 002_add_phone.sql         # ALTER TABLE ... ADD COLUMN  
└─ 003_create_orders.sql     # CREATE TABLE orders
```

Cada migration roda **uma vez**, em ordem.

O banco lembra quais já foram aplicadas.

É o `git` do schema — sem migrations, você não sabe o estado do banco em produção.

Clean Architecture — separar camadas

```
Handler → Repository (interface) → PostgresRepository  
                                           → MemoryRepository (testes)
```

O handler não sabe se está falando com PostgreSQL ou memória.

Isso é o que permite testar sem banco real.

Resumo

Hoje

1. `map` → 3 problemas: volatilidade, race, escala
2. `sqlc`: SQL puro → Go tipado
3. Docker para PostgreSQL local
4. Schema + queries → `sqlc generate`
5. Pool de conexões com `pgxpool`
6. Handler: sem mutex, context, erro explícito

Entregáveis Sprint 2 (19/05)

- API conectada a PostgreSQL com `sqlc`
- Clean Architecture
- 10+ testes automatizados
- Pipeline CI
- Vídeo 8 min
- *(opcional)* `docker-compose.yml`

Próximos passos

- **Lista 3:** ⚠️ prazo prorrogado até **12/05 (ter) às 23:59**
- **Lista 4** (sqlc + Repository pattern): **publicada** — prazo **19/05 (ter)**
- **Sprint 2:** entrega **19/05 (ter)**

Leituras:

- sqlc docs: docs.sqlc.dev
- Go project layout: github.com/golang-standards/project-layout